

Relatório Exploratório - Aula 007 (8 - 16)

Slide 8

O slide apresenta a sobrecarga de operadores: um mesmo símbolo pode representar operações diferentes dependendo dos tipos envolvidos. O +, por exemplo, pode somar números ou concatenar textos.

Em Java, a concatenação depende também da ordem da expressão:

```
System.out.println('a' + 1);      // 98
System.out.println("" + 'a' + 1); // a1
```

O primeiro caso soma o valor numérico do caractere. No segundo, a string vazia faz com que os próximos valores sejam tratados como texto.

Python permite que o programador defina o comportamento de operadores por métodos especiais, como `__add__` e `__mul__`. Assim, uma classe Vetor pode fazer `v1 + v2` somar coordenadas e `v * 3` multiplicar cada coordenada.

O exemplo ajuda a perceber que a sobrecarga pode deixar o código expressivo, mas também pode ocultar comportamentos inesperados. Java não permite criar novos significados para operadores; com `BigDecimal`, por exemplo, é necessário escrever chamadas como `x.add(y).multiply(tres)`.

Slide 9

Este slide trata das conversões de tipos, distinguindo **alargamento**, **estreitamento** e **coerção**.

- Alargamento transforma um tipo menor em outro capaz de representar mais valores, como `int` para `float`.
- Estreitamento pode perder informação, como `double` para `int`.
- Coerção é a conversão automática feita pela linguagem.

Exemplo em Java:

```
int grande = 123456789;
float f = grande;           // conversão implícita
System.out.println((int) f); // 123456792: houve perda de precisão

double d = 3.99;
System.out.println((int) d); // 3
System.out.println((byte) 200); // -56
```

O resultado de 123456789 convertido para `float` e depois de volta para `int` muda porque o tipo intermediário não representa exatamente todos os inteiros. Já o `cast`

para int descarta a parte fracionária, e o cast para byte sofre os efeitos do intervalo limitado do tipo.

Outro detalhe é que operações com byte em Java são promovidas para int:

```
byte b = 10, c = 20;  
// byte soma = b + c; // não compila
```

Também é importante diferenciar divisão inteira de divisão real:

```
System.out.println(1 / 2); // 0  
System.out.println(1 / 2.0); // 0.5
```

Slide 10

O slide compara a coerção implícita de JavaScript com o *type juggling* do PHP. Em JavaScript, o operador usado pode mudar completamente o resultado:

```
"5" - 2; // 3  
"5" + 2; // "52"  
"5" * "2"; // 10
```

O - e o * tentam converter as strings para números, enquanto o + pode escolher a concatenação. Por isso, expressões visualmente parecidas não têm necessariamente a mesma semântica.

As comparações não estritas também são perigosas:

```
0 == ""; // true  
"" == "0"; // false  
0 == "0"; // true  
0 === "0"; // false
```

O fato de `0 == ""` e `0 == "0"` serem verdadeiros, mas `"" == "0"` ser falso, mostra que a igualdade com coerção não é transitiva.

No PHP, hashes que começam com 0 e podem ser interpretados como notação científica e duas strings diferentes podem ser consideradas iguais com `==`. A recomendação prática apresentada é usar comparações estritas (`===` e `!==`) quando a conversão não for desejada.

Slide 11

As expressões relacionais e booleanas mudam bastante entre linguagens. Em C, um booleano é representado por um inteiro: zero significa falso e qualquer valor diferente de zero significa verdadeiro.

```
int a = 3, b = 2, c = 1;
```

```
printf("%d\n", a > b > c);          // 0
printf("%d\n", (a > b) && (b > c)); // 1
```

$a > b > c$ não significa “a maior que b e b maior que c”. A expressão é avaliada da esquerda para a direita: $a > b$ vale 1, e então $1 > c$ é falso.

Python trata comparações encadeadas de outra maneira:

```
a, b, c = 3, 2, 1
print(a > b > c) # True
```

Nesse caso, a expressão equivale a $(a > b)$ and $(b > c)$. O slide também mostra que Python considera valores como 0, "", listas vazias e None falsos, enquanto strings não vazias, inclusive "0", são verdadeiras.

Java, C#, Go e Rust exigem um valor booleano em um if; tentar usar um inteiro diretamente não compila. Essa restrição evita parte das confusões presentes em C, Python e JavaScript.

Slide 12

O curto-circuito permite que a segunda parte de uma expressão seja ignorada quando o resultado já pode ser determinado.

```
while (i < lista.length && lista[i] != chave) {
    i++;
}
```

O && verifica primeiro $i < \text{lista.length}$. Se a condição for falsa, $\text{lista}[i]$ não é acessado, evitando uma posição fora dos limites.

Em Java, && faz curto-circuito, mas & avalia os dois lados:

```
s != null && s.length() > 0; // seguro
s != null & s.length() > 0;  // pode causar NullPointerException
```

O mesmo cuidado vale para efeitos colaterais. Em $(a > b) || (b++ / 3)$, o incremento só ocorre quando $a > b$ é falso. Portanto, a avaliação da expressão também pode alterar o estado do programa.

Em Python, and e or devolvem um dos próprios operandos, e não necessariamente um booleano:

```
print(0 or "padrão") # padrão
```

JavaScript possui ainda ?. , para acesso opcional, e ??, para substituir apenas null ou undefined:

```
qtd || 10; // 10 quando qtd é 0
qtd ?? 10; // 0 quando qtd é 0
```

Slide 13

O slide apresenta diferentes formas de atribuição. Em C, C++ e JavaScript, a atribuição é uma expressão:

```
sum = count = 0;
a = b + (c = d / b) - 1;

while ((ch = getchar()) != EOF) {
    /* ... */
}
```

Essa característica permite atribuições múltiplas e atribuições dentro de condições, mas também facilita erros:

```
if (x = y) {
    /* atribui y a x e testa o resultado */
}
```

O código pode compilar quando a intenção era escrever `x == y`.

Python não permite a atribuição comum dentro de uma expressão, mas possui atribuição múltipla e o operador morsa `:=`:

```
a, b = b, a

if (n := len(dados)) > 5:
    print(n)
```

Go não trata a atribuição nem `x++` como expressões. Rust também impede que uma atribuição accidental seja usada como condição. Linguagens funcionais, como ML e F#, normalmente dão nome a valores sem permitir que eles sejam alterados, reforçando a distinção entre expressão e estado mutável.

Slide 14

A atribuição de modo misto combina atribuição com uma operação. Em Java, a atribuição simples permite alargamento:

```
double d = 3;
long l = 10;
```

O estreitamento precisa de cast explícito:

```
int i = (int) 3.7; // 3
```

Porém, operadores compostos podem esconder uma conversão:

```
int i = 3;
i += 3.7;
System.out.println(i); // 6
```

O equivalente `i = i + 3.7` não seria aceito sem `cast`. Essa diferença é uma armadilha porque o operador abreviado parece apenas uma forma compacta da atribuição longa.

Em C, coerções podem ocorrer em várias direções, inclusive com perda silenciosa:

```
int i = 3.7;           // 3
char c = 300;          // valor depende do intervalo de char
unsigned int u = -1;
```

Outra armadilha clássica é calcular uma porcentagem usando apenas inteiros:

```
double taxa = acertos / total * 100;
```

A divisão inteira acontece antes da conversão para `double`. Uma alternativa é converter um dos operandos antes da divisão, como `100.0 * acertos / total`.

Slide 15

O slide inicia a parte de estruturas de controle e retoma o resultado de Böhm e Jacopini: sequência, seleção e iteração são suficientes para expressar qualquer algoritmo.

- **Sequência:** executar uma sentença após a outra.
- **Seleção:** escolher um caminho com `if` ou uma construção equivalente.
- **Iteração:** repetir enquanto uma condição for satisfeita.

```
a = 1;
b = a + 2;
```

```
if (x > 0) ...
```

```
while (x > 0) {
    x--;
}
```

Construções como `for`, `switch` e `foreach` não são necessárias em termos de poder computacional, mas tornam determinados algoritmos mais fáceis de escrever e ler. O problema aparece quando a linguagem oferece tantas formas diferentes que o leitor já não reconhece rapidamente a estrutura do código.

Slide 16

Este slide apresenta o problema do **else pendente**. Em C, Java e C++, o `else` pertence ao `if` mais próximo que ainda não possui `else`, independentemente da indentação:

```
if (sum == 0)
    if (count == 0)
        result = 0;
```

```
    else
        result = 1;
```

Nesse exemplo, o else pertence ao segundo if. Se sum não for zero, nenhum dos dois comandos é executado e o valor inicial de result permanece. A indentação pode sugerir uma associação diferente da regra real.

A forma segura é usar chaves:

```
if (sum == 0) {
    if (count == 0) {
        result = 0;
    }
} else {
    result = 1;
}
```

Go e Rust exigem chaves, eliminando esse tipo de ambiguidade. O slide também mostra seletores que devolvem valores:

```
let y = if x > 0 { x } else { 2 * x };
```

Em Python, a forma equivalente é uma expressão condicional:

```
y = x if x > 0 else 2 * x
```